

北京工商大学

2023-2024 学年第一学期《Linux 操作系统实践》期末试题 A

(计算机科学与技术专业)

| 题号        | 一 | 二 | 三 | 四 | 五 | 总分 |
|-----------|---|---|---|---|---|----|
| 得分        |   |   |   |   |   |    |
| 评阅人<br>签字 |   |   |   |   |   |    |

一、填空题 (本题共 9 小题, 每小题 2 分, 共 18 分)

| 名字        | 功能           | 参数                                                     | 返回值                                                    |
|-----------|--------------|--------------------------------------------------------|--------------------------------------------------------|
| opendir() | (1) 打开目录文件   | (2) 模式                                                 | (3) 指针<br>DIR*<br>失败返回 NULL                            |
| read()    | (4) 从文件中读取数据 | (5) fd (文件描述符)<br>+<br>buf (缓冲区)<br>+<br>count (读取字节数) | (6) 文件内容指针<br>成功: 返回实际读取的字节数<br>失败: 返回 -1<br>到达文件尾返回 0 |

(7) ls 命令中 -l 的功能是: 显示目录中文件的详细属性

(8) main 函数的输入参数是由谁给定的: 由操作系统在启动程序时, 通过命令行输入的参数和传递给该程序的

(9) cp 命令的功能是: 新建或清空目标文件, 将源文件写入目标文件

二、问答分析题 (本大题共 5 小题, 第 1-4 小题每题 6 分, 第 5 小题 10 分, 共 34 分)

1. 在 Linux 系统编程中, 管道等特殊文件与普通文件 (相同/不同) (区别)

相同点: 它们都以文件的形式存在于系统中, 都有一个文件名, 都能使用 open / read / write / close 等接口进行操作。

不同点: 设备文件: mode 指明设备文件的函数或表示程序, 这些方式决定了读写语义 (str, 设备调用 (ioctl) 和 t(setattr))

普通文件: mode 指明普通数据文件, 具有字节大小属性, 设备文件没有, 所有设备均使用方式不同 (open, fcntl)

针对第一题.

## ① 文件 I/O

|          |         |                       |                 |
|----------|---------|-----------------------|-----------------|
| open()   | 打开文件    | pathname, flags, mode | 成功: fd 失败: -1   |
| read()   | 读文件数据   | fd, buf, count        | ...             |
| writel() | 向文件中写数据 | fd, buf, count        | 实际写入字节数, 失败: -1 |
| close()  | 关闭文件    | fd                    | 成功: 0, 失败: -1   |
| seek()   | 移动文件指针  | fd, offset, whence    | 新偏移量 失败: -1     |

## 2. 目录操作

|          |      |         |                                  |
|----------|------|---------|----------------------------------|
| opendir  |      | dirname | 成功: DIR *, 失败: NULL<br>(目录指针)    |
| readdir  | 读目录项 | DIR *   | 成功: struct dirent *, 失败/结束: NULL |
| closedir | 关闭目录 | DIR *   | 成功: 0, 失败: 1                     |

## 3. 文件属性

|        |       |                         |               |
|--------|-------|-------------------------|---------------|
| stat() | 取文件属性 | pathname, struct stat * | 成功: 0, 失败: -1 |
|--------|-------|-------------------------|---------------|

# 试题 A

2. 请写出 who 命令的逻辑流程，并写出伪代码。

逻辑流程：  
 ① 打开 `/var/run/utmp`  
 ② 读 utmp 中每条记录  
 ③ 判断元数据是否有效  
 ④ 如果是则提取其相关信息  
 ⑤ 打印  
 ⑥ 直到块尾，结束

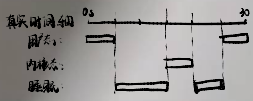
伪代码：  

```

id = open("/var/run/utmp");
while (设备元记录 且 块未读完) {
    if (记录元数据有效) {
        打印相关信息;
    }
}
close(id);
    
```

3. 解释内核提供给用户进程的三个计时器的名称，在时间轴上画出一个进程的三种时间之间的关系。

① TIMER - REAL：真实时间  
 ② TIMER - VIRTUAL：此用户占用时间  
 ③ TIMER - PROF：此用户占用核心时间



从图中可以看出：  
 真实时间约 20s，虚拟时间约 10s，实际时间为 15s

4. pwd 命令的功能是什么，程序思路是什么？

① pwd 的功能是打印当前所在目录的绝对路径

② 其程序思路是逐层递归式来统计绝对路径并打印

1. 打印当前目录名称

2. 取当前目录的 inode 号

3. 判断当前目录文件中 inode 号是否与 inode 号一致，如果一致，则打印根目录

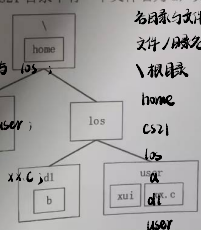
否则：  
 a. 当前目录名称 + inode 号  
 b. 递归父目录  
 c. 打印父目录名称，并递归父目录名称  
 d. 递归父目录名称  
 e. 递归父目录名称 + inode 号

5. 从操作系统的角度，简述下图中各目录之间的连接关系，给出 i 节点编号并画出 i 节点之间的

数据连接关系，i 节点的值可以已定，要求数据一致，能够说明问题。（注：第二排左格子意思是 cs21 目录下有一个文件名为 a，其他的格子也是类似的。）

各目录之间的连接关系：

根目录下有个 home 目录；  
 home 目录下有 2 个目录 cs21 与 los；  
 cs21 目录下有个 a 文件；  
 los 目录下有 2 个目录 dl 与 user；  
 dl 目录下有个 b 文件；  
 user 目录下有 2 个文件 xlc 与 xlc.d



各目录与文件的 i 节点编号如下：

| 文件/目录名 | inode 号 |
|--------|---------|
| \ 根目录  | 1       |
| home   | 2       |
| cs21   | 3       |
| los    | 4       |
| a      | 5       |
| b      | 6       |
| user   | 7       |
| b      | 8       |
| xlc    | 9       |
| xlc.d  | 10      |

节点之间的数据连接关系：

1 指向 2 的目录内容：  
 "home" → 2  
 2 指向 4 的目录内容：  
 "cs21" → 4  
 "los" → 3  
 3 指向 6 的目录内容：  
 "a" → 5  
 "dl" → 6  
 "user" → 7  
 4 指向 5 的目录内容：  
 "a" → 5  
 6 指向 8 的目录内容：  
 "b" → 8  
 7 指向 9 的目录内容：  
 "xlc" → 9  
 "xlc.d" → 10

反正是每个目录都有“.”与“..”两个内容

三、阅读程序，回答下述问题（本题满分12分）

```
#include <sys/types.h>
#include <unistd.h>

main()
{
    int i=0;
    printf("my pid is %d\n", getpid() );
    fork();
    fork();
    fork();
    printf("No. %d, my pid is %d, my ppid is %d\n", i++, getpid(), getppid());
}
```

(1) 该程序执行后，一共会创建几个子进程？（2分）

$2^3=8$ ，子进程有8-1个

(2) 如果得到了下面的结果，解释为什么有5个进程的ppid是1？（2分）

```
my pid is 4041
No. 0, my pid is 4043, my ppid is 4041
No. 0, my pid is 4042, my ppid is 4041
No. 0, my pid is 4044, my ppid is 1
No. 0, my pid is 4046, my ppid is 1
No. 0, my pid is 4041, my ppid is 3861
No. 0, my pid is 4045, my ppid is 1
No. 0, my pid is 4047, my ppid is 1
No. 0, my pid is 4048, my ppid is 1
```

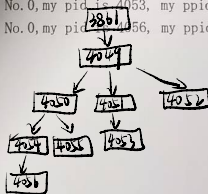
原因及因为 这些进程的 执行是并发的，当一个子进程的父进程比其先执行完了，那么这个进程就变成了孤儿进程，而该进程（即PID为1的进程）会自动收养这些“孤儿”，当这些孤儿进程执行到最后一句时，得到的结果就是PPID为1

(3) 如果想知道所有这些父进程 PID 为 1 的进程的 真正的父进程的 PID，应该如何修改这个程序？ (3 分)

在 fork() 后面添加 wait()，因为 wait() 会让父进程等待，直到他的一个子进程运行结束，再继续运行，所以会运行到子进程最后调用此函数端，它执行了 fork()，所以它直接运行到子进程时，通过它自己可以拿到子进程的 PID。

(4) 如果得到了下面的结果，请你用 进程树 方式，画出各个进程之间的关系图。 (5 分)

```
my pid is 4049
No.0, my pid is 4049, my ppid is 3861
No.0, my pid is 4052, my ppid is 4049
No.0, my pid is 4051, my ppid is 4049
No.0, my pid is 4050, my ppid is 4049
No.0, my pid is 4055, my ppid is 4050
No.0, my pid is 4054, my ppid is 4050
No.0, my pid is 4053, my ppid is 4051
No.0, my pid is 4056, my ppid is 4054
```



四、调用系统调用，写代码段完成下述功能 (本大题共 2 小题，每小题 8 分，共 16 分)

1. 为一个 tty 连接开启字符回显。

```
int main() {
    struct termios tty;
    if (tcgetattr(0, &tty) != 0) {
        perror("tcgetattr");
        return -1;
    }
    tty.c_lflag |= ECHO;

    if (tcsetattr(0, TCSANOW, &tty) != 0) {
        perror("tcsetattr");
        return -1;
    }
    return 0;
}
```

2. 设置一个定时器并设定，循环地从键盘接收一个字符，如果是'f'，就让定时器延时减半；如果是'j'，就让定时器延时增加1倍。

```
struct interval timer;

void alarm_handler(int signal) {
    printf("timeout\n");
}

// 设置定时器
void set_timer(int sec) {
    timer.it_value.tv_sec = sec;
    timer.it_value.tv_usec = 0;
    timer.it_interval.tv_sec = 0;
    timer.it_interval.tv_usec = 0;
    set_timer(ITIMER_REAL, timer, NULL);
}
```

```
int main() {
    char ch;
    int delay = 2; // 初始延时2秒
    signal(SIGALRM, alarm_handler);
    set_timer(delay);

    while(1) {
        read(0, &ch, 1); // 最多读1个字节
        if (ch == 'f') {
            delay /= 2; // 延时减半
        } else if (ch == 'j') {
            delay *= 2; // 延时加倍
        }
        set_timer(delay);
    }
    return 0;
}
```

##### 五、阅读程序，回答问题（本题满分20分）

```
#include <unistd.h>
#define LEFTEDGE 10
#define RIGHTEDGE 30
#define ROW 10
main()
{
    char message[] = "Hello";
    char blank[] = " ";
    int dir = 1; // 初始向右
    int pos = LEFTEDGE;

    initcsr();
    clear();
    while(1) {
        ① move(L, pos);
        ② addstr(message); // park the cursor */
        refresh(); // show string */
        sleep(1);
        ③ move(R, pos); // erase string */
        ④ addstr(blank);
        pos += dir; // advance position */
        if (pos >= RIGHTEDGE) // check for bounce */
            ⑤ dir = -1;
        if (pos <= LEFTEDGE)
```

- ⑥ \_\_\_\_\_;
- (1) 填写空白处的代码；
  - (2) 解释填写代码的功能；
  - (3) 如果想把sleep(1)替换为，完成什么功能
  - (4) 编写新的主程序 main

⑥  $\text{dir} = +1$  ;

- (1) 填写空白处的代码,使字符串能够在屏幕自由移动移动无限制。(6分)
- (2) 解释写死代码的功能和含义。(6分)
- (3) 如果想把 sleep(1)替换成定时器方式实现字符串的自动移动,应该调用哪些系统调用,完成什么功能? (3分)
- (4) 编写新的主程序 main() 定时器信号应该用以及相关的其他函数。(5分)

### 13. 需要使用的系统调用:

(2) ① `move (row, pos)` 移动到第 `row` 行, `pos` 列

```
setTimer( ), signal( ), pause( )
```

② `addstr (message)`: 在当前位置打印 `message`;

系统功能：使用定时器周期性产生 SIGALRM 信号，在信号处理函数中更新字符串位置，从而实现自动换行。

② move (row, pos): 然后移动圆环位置

④ addstr( blank) : 用空白覆盖 message 的内容, 实现删除的效果

④.  $dir = -1$  : 改变移动方向, 向左移动

⑥ dir = +1: 改变方向向右移动

14) 作为处理数据

```
void nerve -msg (int signal) {
```

4. 物探院

```
move( Rnd, pos) ;
splitStr ( blank) ;
```

4 更新位置

```
pos += dir;
```

If (pos >= RMHTFDGE)

$$dx = -1,$$

if ( pos <= LEFTEDGE )

dir = 115

```
move (p0w, p0s);
```

exist (message);

refresh ( );

3

主編

```
int main () {
```

start interval timer

initscr( ),

```
clear( );
```

```
Signal(SIGALRM, move-msg);
```

$$t_{\text{over}} + t_{\text{welling}} + t_{\text{up-sec}} = 1$$

timer, it - value, lv - msec = 0;

```
timer.it_interval.tv_sec = 1;
```

tuner: it - interval, tw - usec = 0.

settimer (TIMER\_REAL, &amp;timer, NULL);

White 1)

print( )

endwin ();

return 0;

3

